

DS	Session	Hashing	Question Information	Level 1	Challenge 97
----	---------	---------	----------------------	---------	--------------

Problem Description

Given an array A of N integers. Now, you have to output the sum of unique values of the maximum subarray sum of all the possible subarrays of the given array A .

Note: Subarray means contiguous elements with at-least one element in it.

Constraints

$1 \leq N \leq 2000$
 $0 \leq |A_i| \leq 10^9$

Input Format

The first line of the input contains a single integer N , the total number of elements in array A .
The next line of the input contains N space-separated integers representing the elements of the array.

Output Format

The only single line of the output should contain a single integral value representing the answer to the problem.

Logical Test Cases

Test Case 1

INPUT (STDIN)

4
5 -2 7 -3

EXPECTED OUTPUT

17

Test Case 2

INPUT (STDIN)

15
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9

EXPECTED OUTPUT

2021

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
int NA[N];
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

555

Test Case 3

NLOC

73

Hashing7.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int cmp_ll(const void *a, const void *b) {
5     long long v1 = *(const long long *)a;
6     long long v2 = *(const long long *)b;
7     if (v1 < v2) return -1;
8     if (v1 > v2) return 1;
9     return 0;
10}
11
12int main() {
13     int N;
14     if (scanf("%d", &N) != 1) return 0;
15
16     int NA[N] ;
17     for (int i = 0; i < N; i++) {
18         scanf("%d", &NA[i]);
19     }
20
21     long long total_subs = (long long)N * (N + 1) / 2;
22     long long *max_sums = (long long *)malloc(total_subs * sizeof(long long));
23     long long count = 0;
24
25     for (int l = 0; l < N; l++) {
```

```
26     long long max_so_far = NA[1];
27     long long curr_max = NA[1];
28     max_sums[count++] = max_so_far;
29
```

main | CH.SC.U4CSE25225 | Hashing7.c Ln 57, Col 1 Spaces: 4 UTF-8 CRLF C

```
30     for (int i = 1 + 1; i < N; i++) {
31         if (curr_max + NA[r] > NA[r]) {
32             curr_max = curr_max + NA[r];
33         } else {
34             curr_max = NA[r];
35         }
36         if (curr_max > max_so_far) {
```

```
37         max_so_far = curr_max;
38     }
39     max_sums[count++] = max_so_far;
40 }
41 }
42
43 qsort(max_sums, count, sizeof(long long), cmp_ll);
44
45 long long total_unique_sum = 0;
46 for (long long i = 0; i < count; i++) {
47     if (i == 0 || max_sums[i] != max_sums[i - 1]) {
```

```
48         total_unique_sum += max_sums[i];
49     }
50 }
51
52 printf("%lld\n", total_unique_sum);
53
54 free(max_sums);
55 return 0;
56}
57
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4

5 -2 7 -3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing7.exe

17

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

15

2 3 4 5 6 7 3 4 2 7 6 8 6 4 9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing7.exe

2021

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>